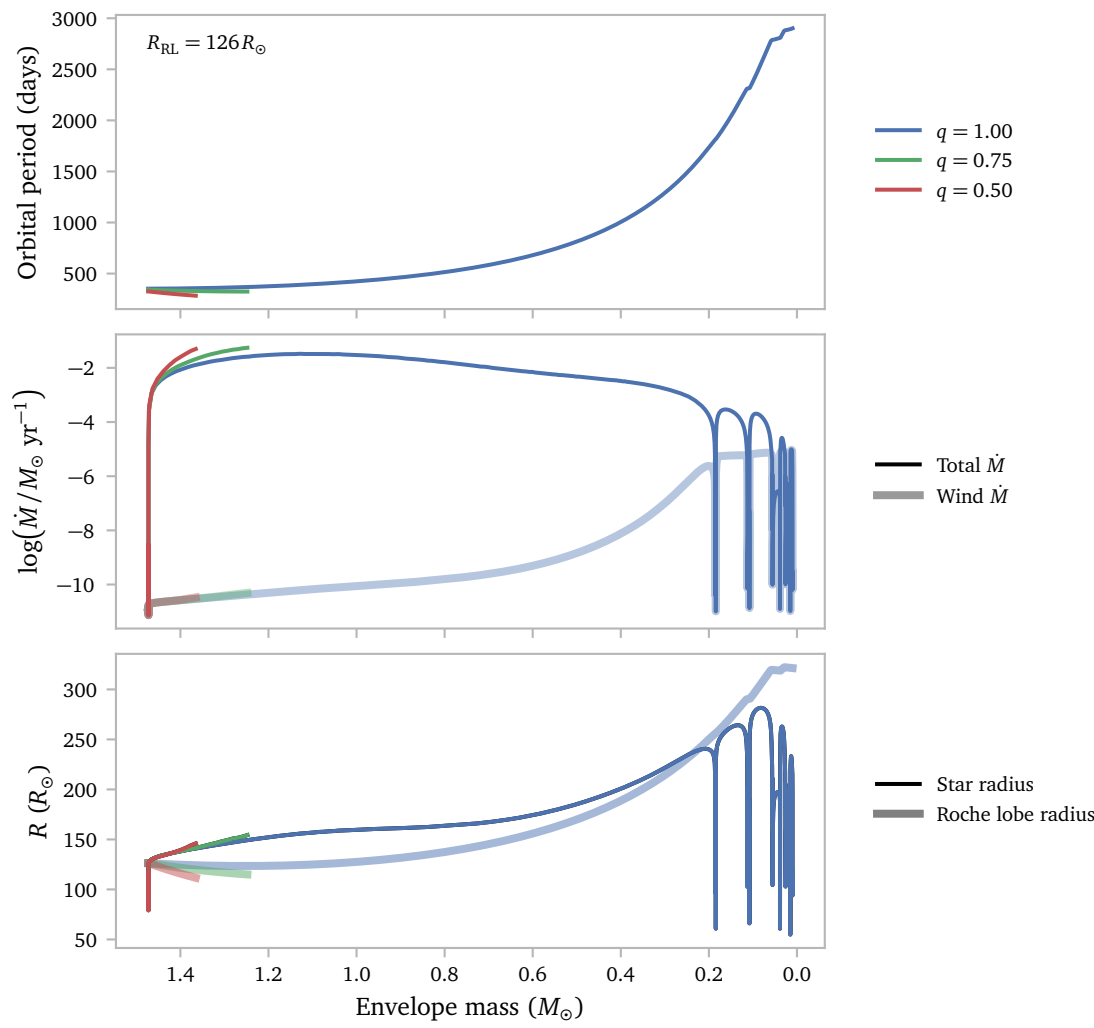


Notes Week 6

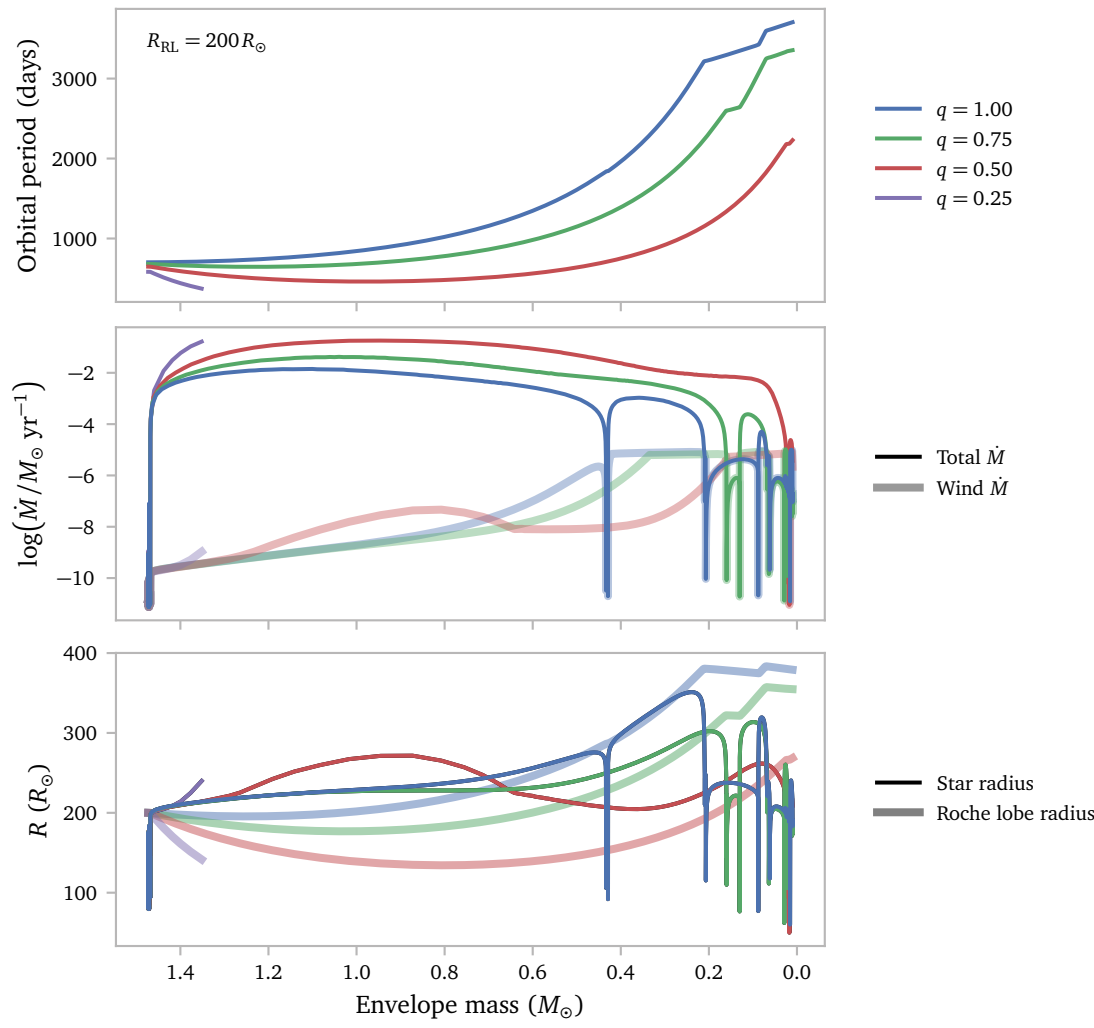
a. Mass coordinate

Instead of plotting relevant stellar and binary properties as a function of time / age, one can plot them against the remaining envelope mass as well. Plotting the properties like this, makes it much easier to see what is actually happening during the most dominant phases of mass transfer.

Below, I plot the same models that I showed last week, but this time with the envelope mass on the horizontal axis.

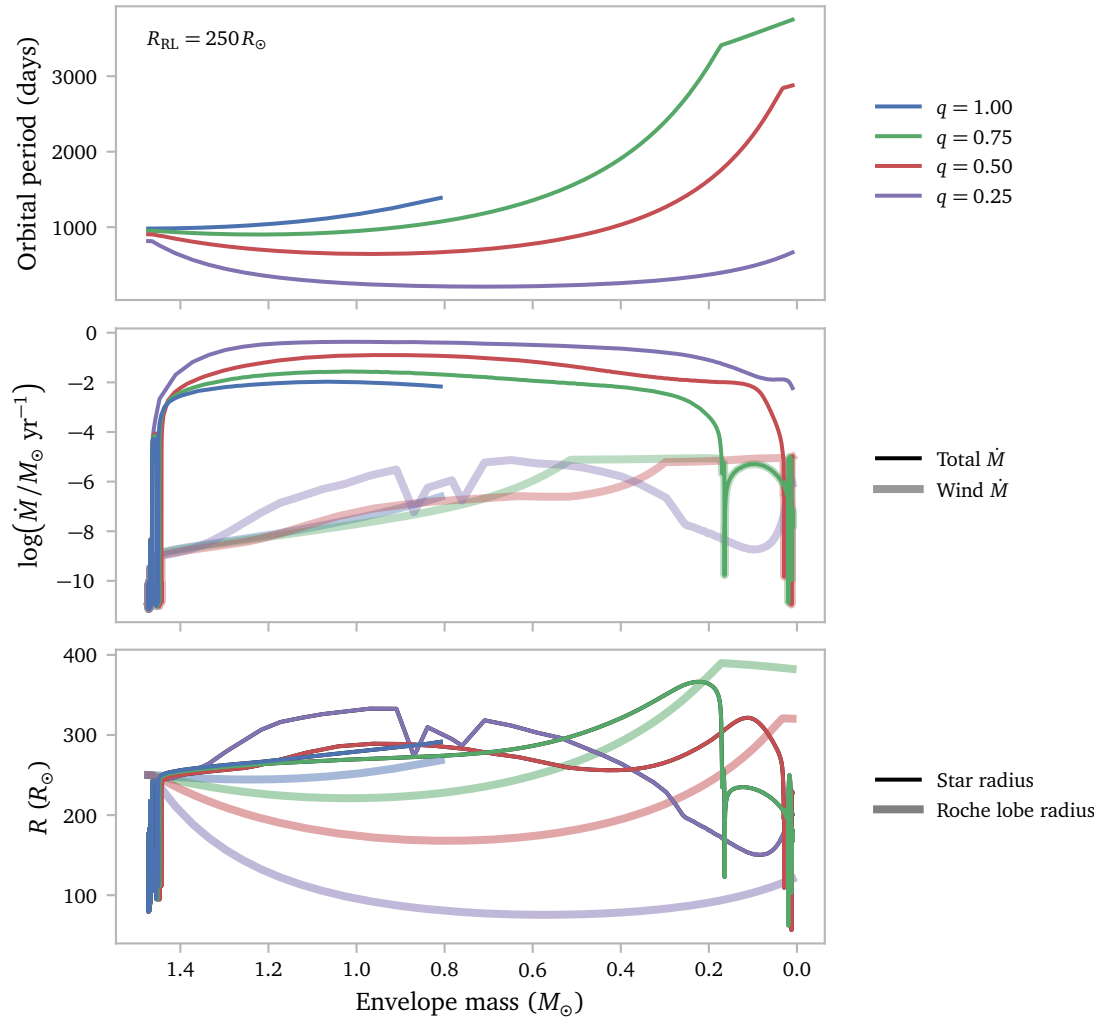


This figure shows that nearly all of the mass is lost due to mass transfer, except for maybe the last couple of $0.01 M_\odot$ of envelope. By plotting the envelope mass on the horizontal axis, it is much easier to compare the star radius with the Roche lobe radius during the mass transfer episodes. For the $q = 1$ model, the stellar radius stays relatively close to Roche lobe radius.

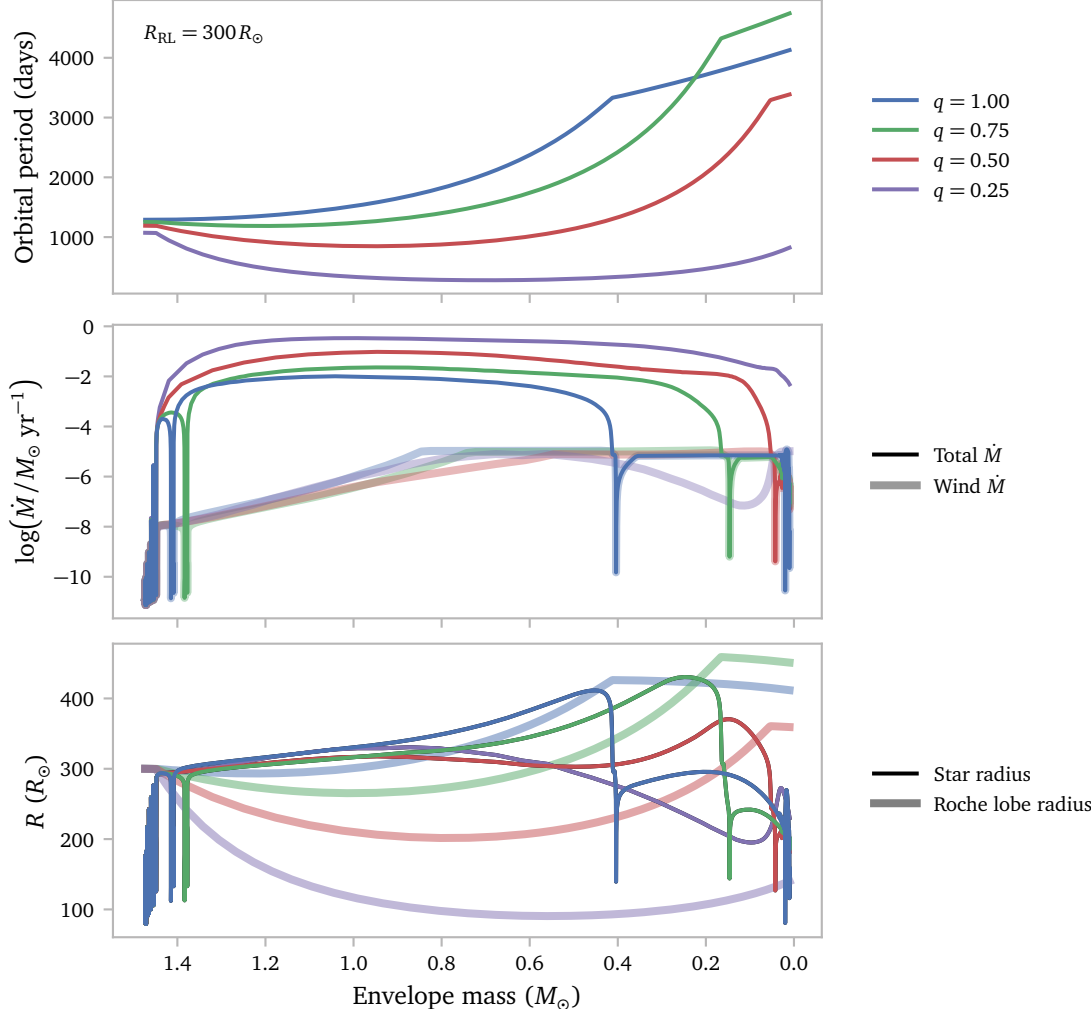


This figure starts to show clear effects of the mass ratio. These can be explained by mass-ratio equalising effects that occur in binaries with unequal mass ratios with fully conservative mass transfer. Most importantly, the Roche lobe radius and orbital period shrink. These effects cause the star to lose more mass.

The significant bump in radius for the $q = 0.5$ model is interesting. I am not sure what causes this.



This plot tells a similar story. Again, there is a significant bump in radius for the low q models, with the $q = 0.25$ model also showing big changes between steps.



Here, the bumps are not visible. This leads me to believe they might be nonfysical.

b. *ÆSOPUS*

To check the details of the *ÆSOPUS* table that MESA uses, I added the following line to the *kap* module of my inlist:

```
show_info = .true.
```

Below, I show the output:

```
read filename </home/koen/mesa-r23.05.1//data/kap_data/AESOPUS_AGSS09.h5>
AESOPUS composition parameters
  Zsun = 1.337E-02
  fCO_ref = -0.385
  fC_ref = -0.752
  fN_ref = -1.286

AESOPUS logT and logR range (logR = logRho - 3 * logT + 18)
  num logTs = 42
    logTs = 3.200 3.220 3.240 3.260 3.280 3.300 3.320 3.340
            3.360 3.380 3.400 3.420 3.440 3.460 3.480 3.500 3.520
            3.540 3.560 3.580 3.600 3.620 3.640 3.660 3.680 3.700
            3.750 3.800 3.850 3.900 3.950 4.000 4.050 4.100 4.150
            4.200 4.250 4.300 4.350 4.400 4.450 4.500
  num logRs = 19
    logRs = -8.000 -7.500 -7.000 -6.500 -6.000 -5.500 -5.000 -4.500
            -4.000 -3.500 -3.000 -2.500 -2.000 -1.500 -1.000 -0.500
            0.500 1.000

AESOPUS metallicities
(These Z values are the reference metallicities)
  num Zs = 13
    Zs = 1.000E-05 3.000E-05 1.000E-04 3.000E-04 1.000E-03
          2.000E-03 3.000E-03 5.000E-03 1.000E-02 2.000E-02
          3.000E-02 5.000E-02 1.000E-01

Table 0.000010: Z = 1.000E-05
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 0.170 0.270 0.290 0.310 0.330 1.350 2.520 3.700
  num fCs = 5
    fCs = -1.000 0.300 1.600 2.900 4.200
  num fNs = 5
    fNs = 0.000 1.170 2.350 3.520 4.700

Table 0.000030: Z = 3.000E-05
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 0.060 0.270 0.290 0.310 0.330 1.110 2.170 3.220
  num fCs = 5
    fCs = -1.000 0.180 1.360 2.540 3.720
  num fNs = 5
    fNs = 0.000 1.060 2.110 3.170 4.220

Table 0.000100: Z = 1.000E-04
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.080 0.270 0.290 0.310 0.330 0.850 1.770 2.700
  num fCs = 5
    fCs = -1.000 0.050 1.100 2.150 3.200
  num fNs = 5
    fNs = 0.000 0.920 1.850 2.770 3.700

Table 0.000300: Z = 3.000E-04
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.190 0.270 0.290 0.310 0.330 0.610 1.420 2.220
  num fCs = 5
    fCs = -1.000 -0.070 0.860 1.790 2.720
  num fNs = 5
    fNs = 0.000 0.810 1.610 2.420 3.220

Table 0.001000: Z = 1.000E-03
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.330 0.270 0.290 0.310 0.330 0.350 1.020 1.700
  num fCs = 5
    fCs = -1.000 -0.200 0.600 1.400 2.200
  num fNs = 5
    fNs = 0.000 0.670 1.350 2.020 2.700

Table 0.002000: Z = 2.000E-03
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.400 0.200 0.270 0.290 0.310 0.330 0.800 1.400
  num fCs = 5
    fCs = -1.000 -0.280 0.450 1.170 1.900
  num fNs = 5
    fNs = 0.000 0.600 1.200 1.800 2.400

Table 0.003000: Z = 3.000E-03
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.440 0.110 0.270 0.290 0.310 0.330 0.670 1.220
  num fCs = 5
    fCs = -1.000 -0.320 0.360 1.040 1.720
  num fNs = 5
    fNs = 0.000 0.560 1.110 1.670 2.220

Table 0.005000: Z = 5.000E-03
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.500 0.000 0.270 0.290 0.310 0.330 0.500 1.000
  num fCs = 5
    fCs = -1.000 -0.380 0.250 0.880 1.500
  num fNs = 5
    fNs = 0.000 0.500 1.000 1.500 2.000

Table 0.010000: Z = 1.000E-02
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.580 -0.150 0.000 0.270 0.290 0.310 0.330 0.700
  num fCs = 5
    fCs = -1.000 -0.450 0.100 0.650 1.200
  num fNs = 5
    fNs = 0.000 0.420 0.850 1.270 1.700

Table 0.020000: Z = 2.000E-02
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.650 -0.300 0.050 0.270 0.290 0.310 0.330 0.400
  num fCs = 5
    fCs = -1.000 -0.530 -0.050 0.420 0.900
  num fNs = 5
    fNs = 0.000 0.350 0.700 1.050 1.400

Table 0.030000: Z = 3.000E-02
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.690 -0.390 -0.080 0.220 0.270 0.290 0.310 0.330
  num fCs = 5
    fCs = -1.000 -0.570 -0.140 0.290 0.720
  num fNs = 5
    fNs = 0.000 0.310 0.610 0.920 1.220

Table 0.050000: Z = 5.000E-02
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.750 -0.500 -0.250 0.000 0.270 0.290 0.310 0.330
  num fCs = 5
    fCs = -1.000 -0.620 -0.250 0.120 0.500
  num fNs = 5
    fNs = 0.000 0.250 0.500 0.750 1.000

Table 0.100000: Z = 1.000E-01
  num Xs = 5
    Xs = 0.400 0.500 0.600 0.700 0.800
  num fCOs = 9
    fCOs = -1.000 -0.830 -0.650 -0.480 -0.300 0.270 0.290 0.310 0.330
  num fCs = 5
    fCs = -1.000 -0.700 -0.400 -0.100 0.200
  num fNs = 5
    fNs = 0.000 0.170 0.350 0.520 0.700

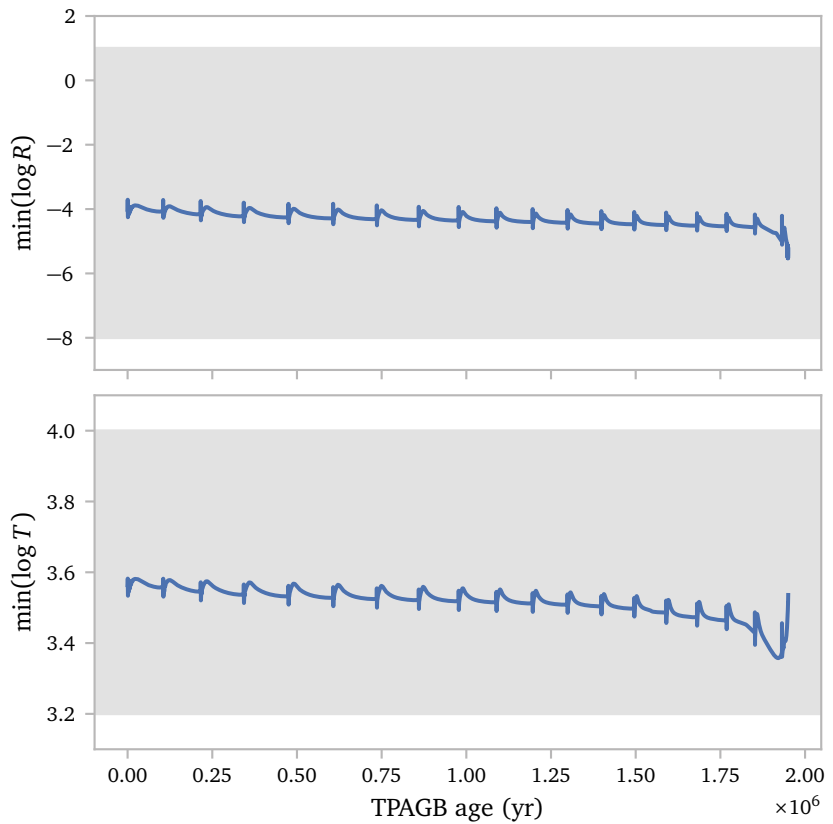
Finished reading AESOPUS tables
```

Most importantly, this table ranges from $\log T = 3.2$ (1585 K) to $\log T = 4.5$ (31622 K) and from $\log R = \log(\rho/(T/10^6 \text{ K})^3) = -8$ to $\log R = 1$. This is the full range *ÆSOPUS* 1.0 (Marigo & Aringer, 2009) and *ÆSOPUS* 2.0 table Marigo et al. (2024b). However, the most recent *ÆSOPUS* 2.1 table (Marigo et al., 2024a), which was released after *MESA* r23.05.1, has an extended low temperature range going down to $\log T = 2$.

ÆSOPUS 2.1 tables can be generated and downloaded from their website: https://stev.oapd.inaf.it/cgi-bin/aesopus_2.1¹.

Once downloaded, these tables can be used to create opacity tables for *MESA*. The file \$MESA_DIR/kap/preprocessor/ÆSOPUS/README describes how this should be done.

Below, I plot the minimum R and T of the standard $2M_\odot$ TPAGB star that I used to generate the models.



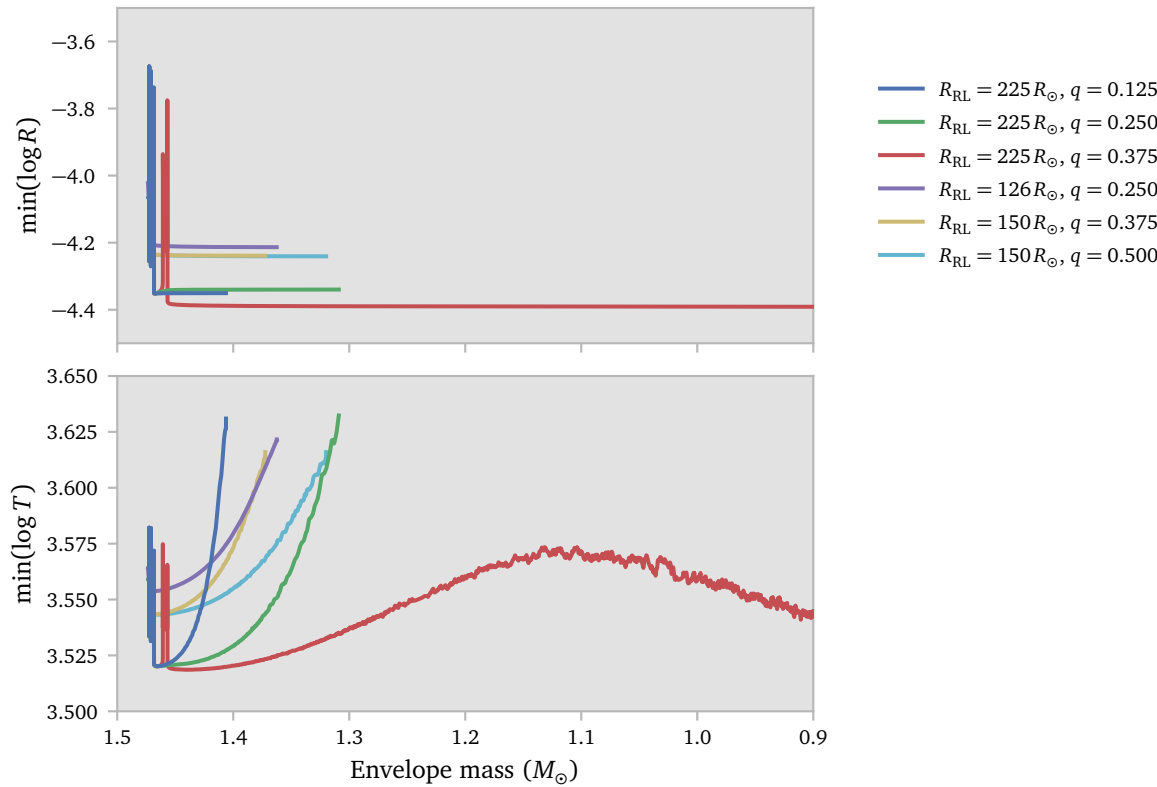
¹ I tried to generate and download some tables, but I was unable to generate tables with multiple f_{CO} , f_{C} and f_{N} values. This makes it very difficult to download all of the required tables.

Clearly, the minima stay well within the table bounds.

The maxima should not be important, since here, the high- T opacity tables will be used.

It is good news that even at its most extreme, the star is still well within the *ÆSOPUS* bounds. However, most problems were encountered in a binary simulation.

Below, I plot a couple of the binaries that displayed the *ÆSOPUS* out of bounds message (spoiler), as well as a model that successfully terminated.



Surprisingly, none of the models temperatures or densities anywhere near the table minima, and seem to even be moving towards more stable configurations before abruptly stopping. I think it is interesting that the successful model seems transition to larger envelope temperatures much more gradually than the models that terminate. The maximum reached is also much lower.

So what is going on?

c. Binary problems

In this section, I explore the problems with the models that terminate early.

To start, let me write an example of the terminal output below:

```
AESOPUS_interp: logR, logT outside of table bounds
AESOPUS_interp: logR, logT outside of table bounds
AESOPUS_interp: logR, logT outside of table bounds
AESOPUS_interp: logR, logT outside of table bounds
retry: adjust_correction failed in eval_equations -- give up in solver 3568
AESOPUS_interp: logR, logT outside of table bounds
AESOPUS_interp: logR, logT outside of table bounds
...
AESOPUS_interp: logR, logT outside of table bounds
AESOPUS_interp: logR, logT outside of table bounds
retry: adjust_correction failed in eval_equations -- give up in solver 3568
retry: logT < hydro_mtx_min_allowed_logT 68 3568
dt 6.9635367095860718D-07
i min_timestep_limit 9.999999999999995D-07
stopping because of problems dt < min_timestep_limit
```

Although the output *looks* like it fails due to problems with the opacity module, this is not the *true* problem.

By digging a little deeper, and enabling `report_solver_progress = .true.` we can see that the model actually fails during the Newton solver step. Below, I show part of this output.

[illegible]

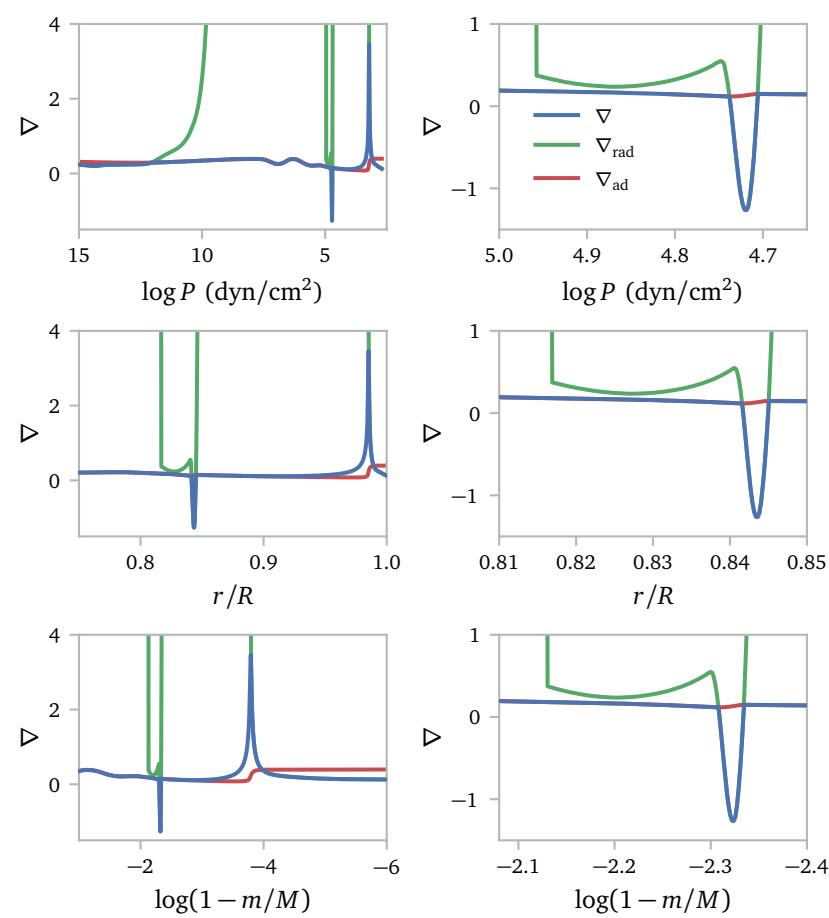
This is quite long, but the most important part to note is that the solver does not seem to converge. This happens both in the dv/dt and $dlnE/dt$ equations.

It is possible to inspect the individual results from the equations for every solver call and zone. I am not sure how to proceed, but below I show at least one problematic zone:

	eql	name dlnf,d							
d_dlnf_dtk(d_lnf=1)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=2)	199	lg rel diff	-8.37666	lg uncertainty	-4.72797	Analytic	1.426604739900E-04	Numeric	1.426604739900E-04
d_dlnf_dtk(d_lnf=3)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=4)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=5)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=6)	199	lg rel diff	-2.17428	lg uncertainty	-8.00876	Analytic	4.205584761300E-04	Numeric	-6.47474283161E-02
d_dlnf_dtk(d_lnf=7)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=8)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=9)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnf=10)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=1)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=2)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=3)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=4)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=5)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=6)	199	lg rel diff	-7.75482	lg uncertainty	-7.83559	Analytic	3.58571220221E-18	Numeric	-3.58571228391E-18
d_dlnf_dtk(d_lnk=7)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=8)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=9)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00
d_dlnf_dtk(d_lnk=10)	199	lg rel diff	-99.00000	lg uncertainty	-99.00000	Analytic	6.000000000000E+00	Numeric	6.000000000000E+00

There seems to be a problem with the $d_{\ln E_{dt}}(k)/d_{\ln T}(k)$ equation.

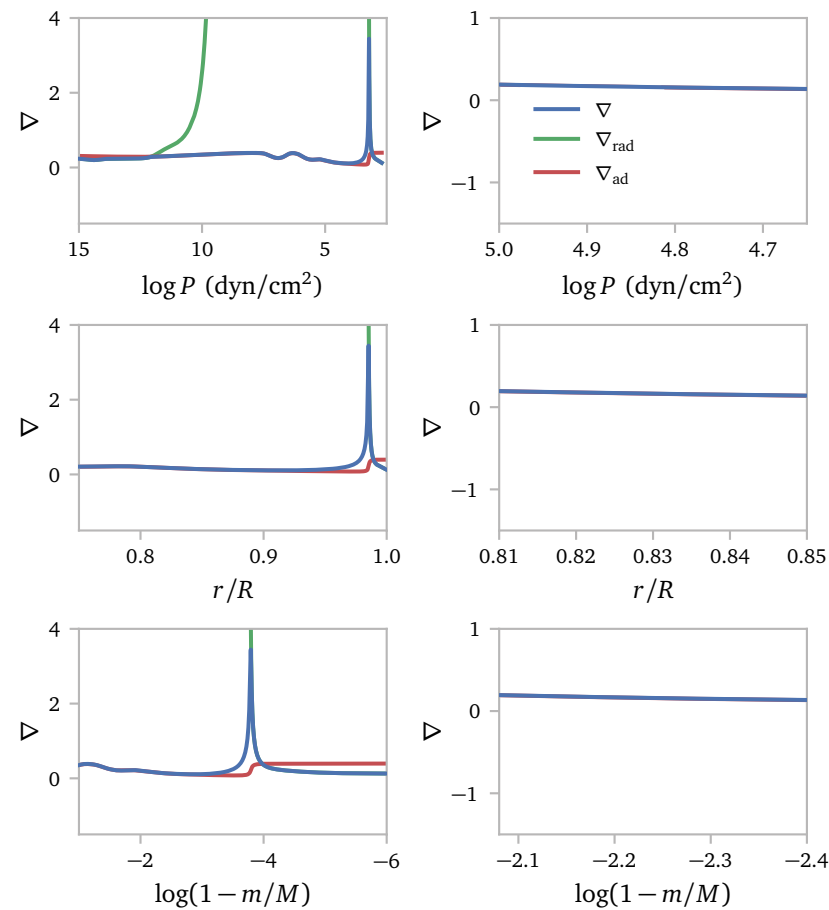
Here, I show the last temperature gradient profile of a model ($R_{\text{RL}} = 126, q = 0.5$) just before it failed very early on during the mass transfer episode.



There is an unexpected dip in the temperature gradient in the convective envelope. This dip is caused by a broader dip in the radiative temperature gradient, and very locally, the envelope actually becomes radiative instead of convective.

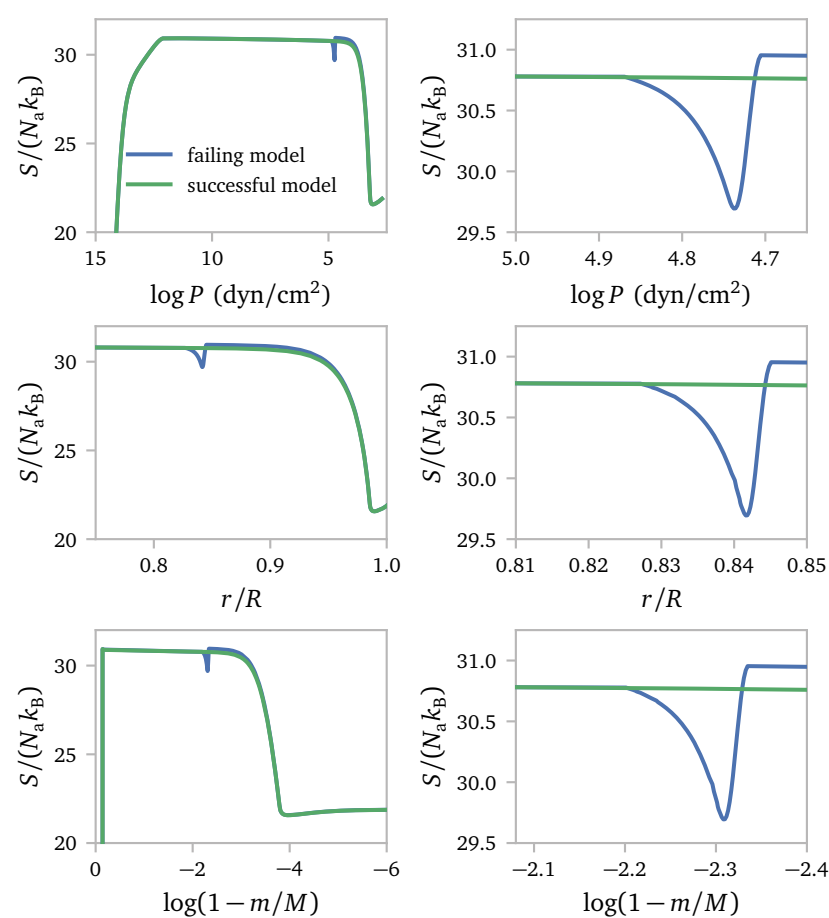
The temperature gradient actually becomes negative in these regions. To me, this seems very problematic.

This unstable situation develops during 1 or 2 successful model steps. Just before this instability occurs, the profiles look perfectly normal (at least to me). Below, I show the profile of the model just before the model failed.



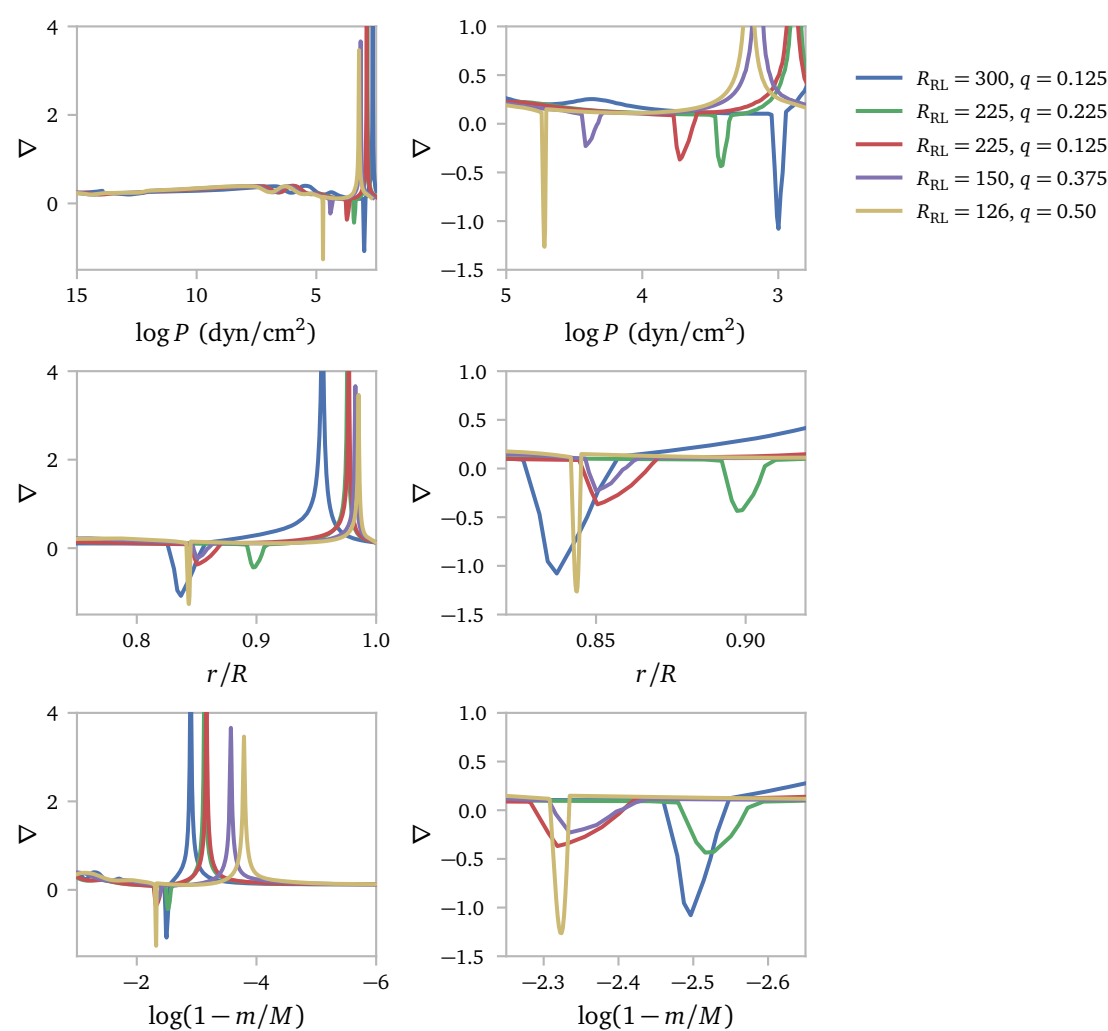
Clearly, the unexpected dip is not present here.

I wanted to check if the problems had something to do with the superadiabatic nature of the outer layers of the star. Here, I show the entropy profile for the models:



The same dip is present here at the same location in the star.

However, as can also clearly be seen, this is *not* in the superadiabatic regions of the star, but in the "normal" convective region.



d. Refactor

Since I am using the `run_star_extras` code from [Rees et al. \(2024\)](#) and implementing my own pieces of code, I wanted to clean up the current extra files that I am using.

I split the code up into multiple files, making it much more clear what belongs together.

The `src` folder now looks like this:

```
- src
|- inc
|  |- additional_routines.inc
|  |- params.inc
|  |- rees2024_agb_routines.inc
|  |- rees2024_mixing_routines.inc
|  |- run_star_extras.inc
|- binary_run.f90
|- run_binary_extras.f90
|- run_star_extras.f90
```

This makes it much easier to find the relevant pieces of code.

In the `additional_routines`, I implement all my custom routines, for example, the routines for saving models at specific radii.

I also implemented code to save the maximum and minimum temperature and $R = \rho/(T/10^6 \text{ K})^3$. Lastly, I save the quasi-adiabatic critical mass loss rate as well.

e. New models

I started some new simulations. Most of them are still running:

678480_[21-63%16]	proj_stev	run-bina	kessers	PD	0:00	1 (Resources)
678480_20	proj_stev	run-bina	kessers	R	1:27:39	1 coma43
678480_19	proj_stev	run-bina	kessers	R	1:31:53	1 coma45
678480_18	proj_stev	run-bina	kessers	R	15:20:12	1 coma44
678480_13	proj_stev	run-bina	kessers	R	1-20:20:39	1 coma43
678480_12	proj_stev	run-bina	kessers	R	1-20:31:40	1 coma44
678480_11	proj_stev	run-bina	kessers	R	1-20:40:35	1 coma43
678480_5	proj_stev	run-bina	kessers	R	1-21:08:39	1 coma44
678480_6	proj_stev	run-bina	kessers	R	1-21:08:39	1 coma45
678480_7	proj_stev	run-bina	kessers	R	1-21:08:39	1 coma45

This has two reasons.

1. I made a typo in my SLURM job script
2. I changed some inlist settings.

e.1 SLURM typo

The previous array job that I submitted used run directories with the names `{NAME}_{index}`. This is fine, but not very clear when modifying both q and R_{RL} , and later possibly other values as well.

Therefore, I changed the run directories to more descriptive names, and I had to change the SLURM script to account for this, so I wrote this:

```
#!/bin/bash
#SBATCH --mail-user=koen.essers@ru.nl
#SBATCH --mail-type=BEGIN,END,FAIL,REQUEUE
#SBATCH --output=output_slurm/R-%x_%A_%a.out
#SBATCH --error=output_slurm/R-%x_%A_%a.err
#SBATCH -p proj_stev
#SBATCH --mem=8G
#SBATCH -N 1 -n 8
#SBATCH --time=0-48:00:00

idx=${SLURM_ARRAY_TASK_ID}
MODEL_DIRS=( $(ls -d
  - /vol/astro8/onnop/kessers/tpagb-mass-transfer/
  - mesa-models/binary-tpagb-grid-2/runs/*) )
WORK_DIR=${MODEL_DIRS[$idx]}
```

This typo caused the `WORK_DIR` to be empty, which is very bad, because it is later used to copy all files in the directory to the cluster node.

After some time I realised that things were not progressing as they should and I realised my mistake.

The empty `WORK_DIR` meant that SLURM / bash thought the `WORK_DIR` was the cluster root directory, and *A LOT* of data was copied to `proj_stev` cluster nodes scratch storage.

I sent an email to Simon and he cleaned up the scratch for me, after which I could actually start simulating.

e.2 Inlist changes

I wanted to modify my inlist settings for a couple of reasons. I wanted to

1. prevent RLOF solver instabilities,
2. resolve mass loss more accurately,
3. relax the termination condition,

I did this by increasing the total number of retries that the solver is allowed to make, such that it can (hopefully) find an acceptable RLOF mass transfer value to prevent the failure that I saw last week:

```
redo_limit = 300
retry_limit = 300
```

I also added a maximum change in mass criterion to the star, similar to what [Temminck et al. \(2023\)](#) did:

```
delta_lg_star_mass_limit = 1.0d-4
delta_lg_star_mass_hard_limit = 1.0d-3
```

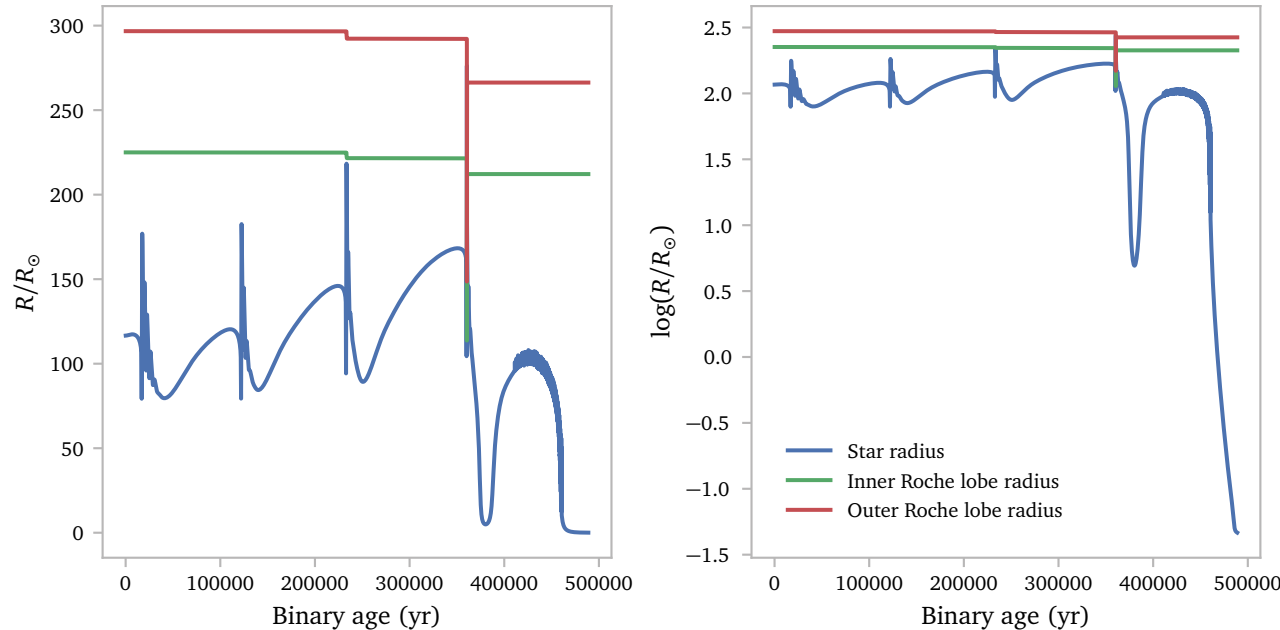
Lastly, I relaxed the minimum envelope mass to:

```
envelope_mass_limit = 0.0001
```

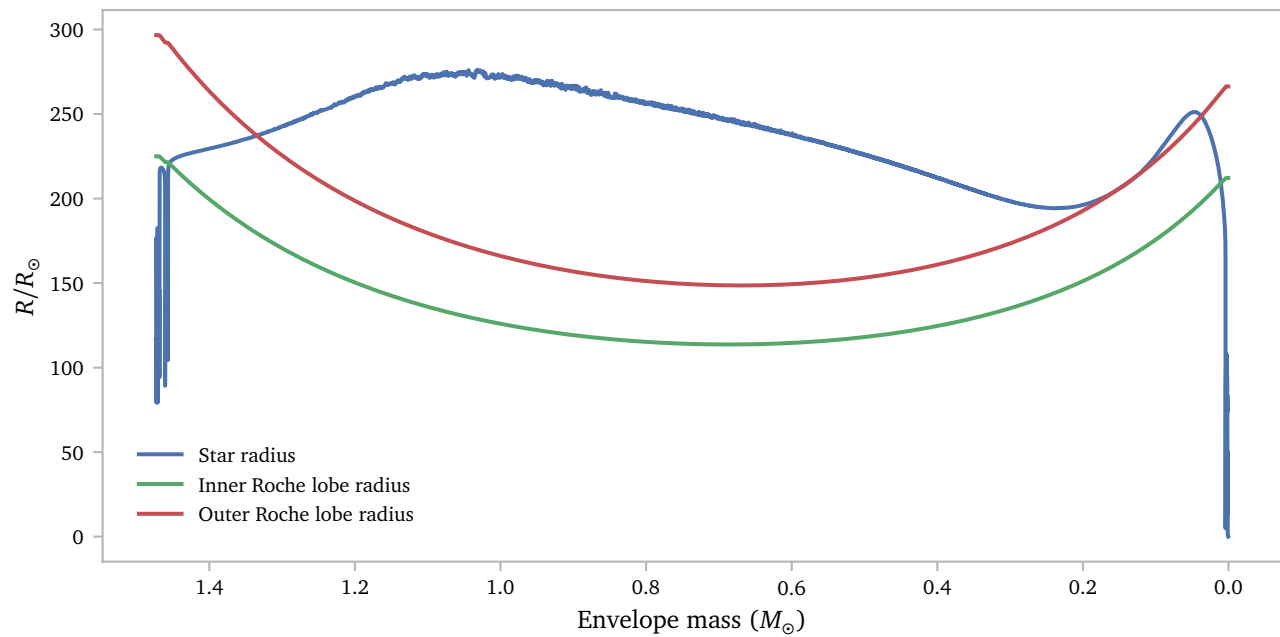
I think especially the last two conditions cause the models to take significantly longer to terminate, with the bulk of the computational time being used to transition to a white dwarf.

I was only able to analyse one of the stars, since only the other models failed, or are still running.

Below, I make some plots of the $R = 225.00 R_{\odot}$ and $q = 0.375$ model, starting with plotting the radius.

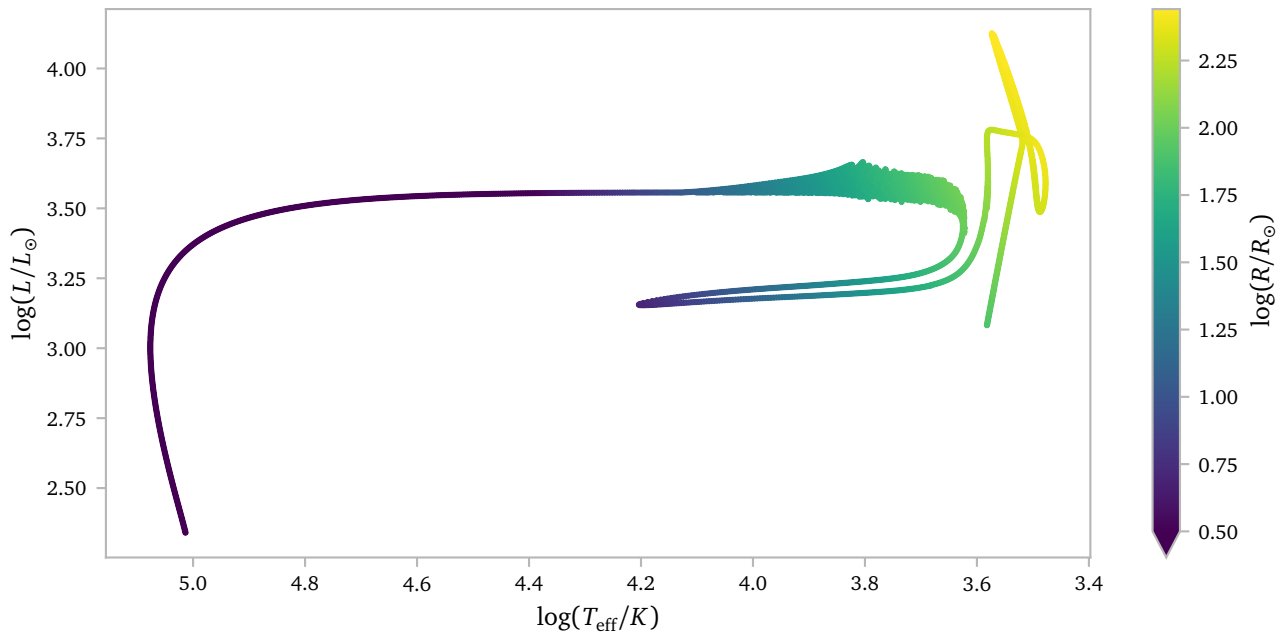


The models from last week did not show the last "bump" and drop to very small (white dwarf) radii.



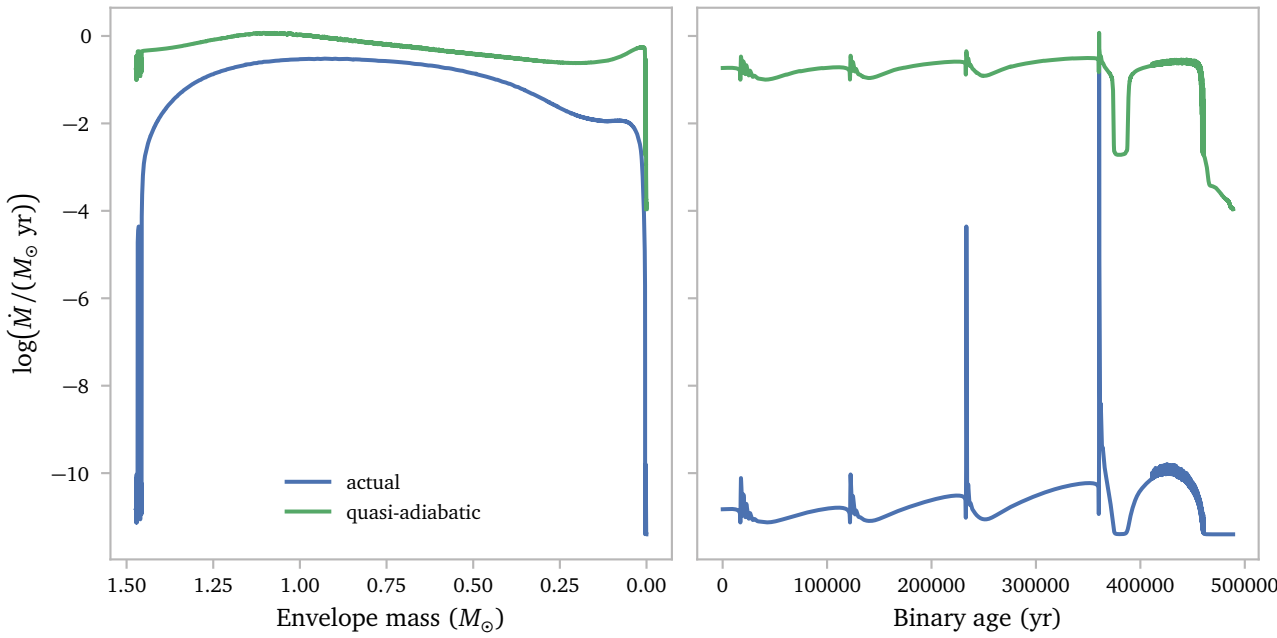
Clearly, the model extends past the outer Roche lobe very early on in its dominant mass transfer episode.

To show the transition of the star to a white dwarf, I plot the HR diagram.

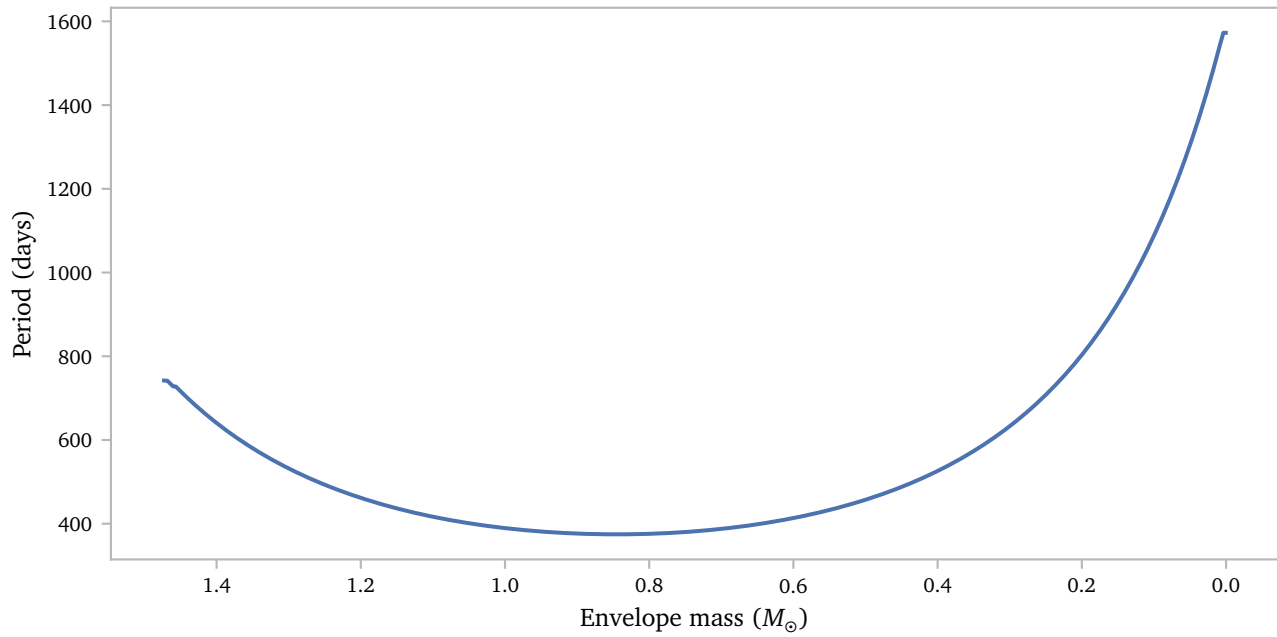


Interestingly, there is a loop just before the final path to a white dwarf. It aligns with the "bump" that was visible in the plots above.

Below, I plot the mass loss rate and the quasi-adiabatic critical mass loss rate from [Temmink et al. \(2023\)](#).



Clearly, the model is below this critical rate all of the time. Lastly, I plot the period.



References

Marigo, P., Addari, F., Bossini, D., et al. (2024)a *ÆSOPUS 2.1: Low-temperature Opacities Extended to High Pressure*. [ApJ976](#), 39.

Marigo, P. & Aringer, B. (2009) *Low-temperature gas opacity. ÆSOPUS: a versatile and quick computational tool*. [A&A508](#), 1539.

Marigo, P., Woitke, P., Tognelli, E., et al. (2024)b *ÆSOPUS 2.0: Low-temperature Opacities with Solid Grains*. [ApJ960](#), 18.

Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. [MNRAS527](#), 9643.

Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2023) *Coping with loss. Stability of mass transfer from post-main-sequence donor stars*. [A&A669](#), A45.

f. Additional plots

